

UNIVERSITÀ DEGLI STUDI DI SALERNO

*DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED
ELETTRICA E MATEMATICA APPLICATA*



Software Architecture Design: Software Architecture Document

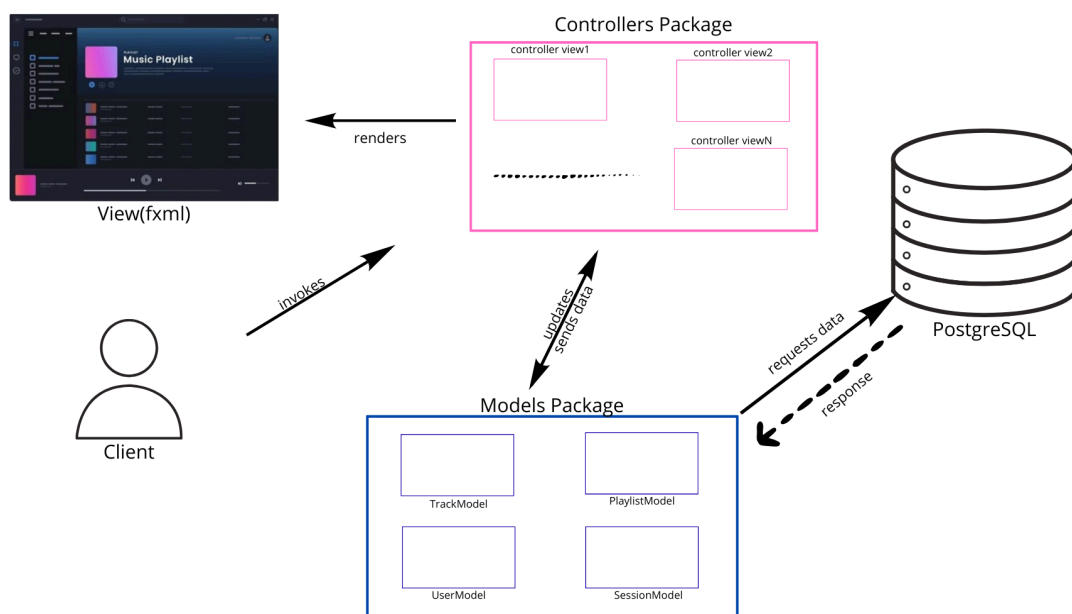
Group Members:

Ragone Vincenzo	IE22700030
Quaranta Davide	IE22700093
Umberto Scassillo	IE22700098
Edmondo Battipaglia	IE22700231

Architettura Iniziale

Il presente documento descrive l'architettura software progettata. Il sistema adotta il pattern architetturale MVC con persistenza dei dati su database PostgreSQL. L'interfaccia utente è definita tramite file FXML e la struttura complessiva del progetto è organizzata in tre package principali:

- controllers: componenti responsabili di aggiornare e usare i model per aggiornare la view
- models: componenti responsabili del mantenimento dello stato dell'applicazione e dei dati del sistema
- views: componenti fxml per l'interfaccia grafica

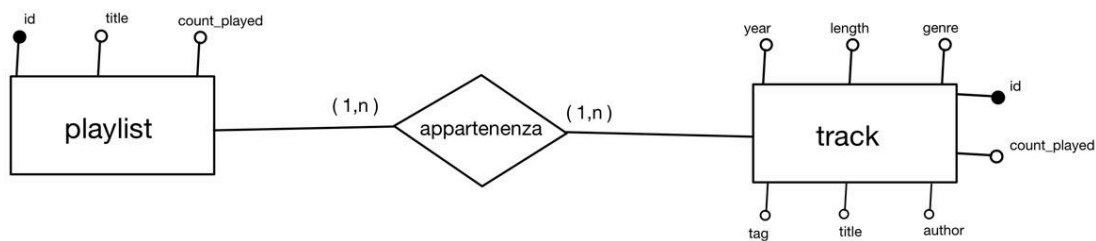


Stack tecnologico adottato

- Linguaggio: Java con sdk 17
- UI framework: JavaFX con integrazione di librerie UI: AtlantaFX e Ikonli
- Database: PostgreSQL
- Test: JUnit Project
- Versioning: Git / Github
- Project Management: Trello

Progettazione e Schema del DB

Lo schema del db è stato prodotto realizzando prima un diagramma concettuale delle entità e delle associazioni richieste dall'applicazione e dai suoi casi d'uso. Il diagramma concettuale risultante è il seguente:



A partire dallo schema concettuale di cui sopra si è passati alla sintesi logica che ha prodotto come risultato le seguenti tabelle:

- Track(id , genre , length , year , tag , title , author , count_played)
- Playlist(id , title , count_played)
- Track_Playlist(playlist_id , track_id)

Alcune considerazioni:

- Per la tabella track si è scelto di utilizzare come identificatore un id univoco seriale per evitare anomalie quali track con lo stesso titolo (se avessimo scelto title come chiave primaria della tabella)
- L'attributo *count_played* è stato inserito per poter facilmente ottenere le tracce e le playlist riprodotte più frequentemente
- Track_Playlist è la tabella che associa ciascuna track ad una playlist. Ci è utile per trovare tutte le track di una determinata playlist. Nella tabella non ci può essere un record con stessa playlist e stessa track ripetuta in quanto la chiave primaria è il combinato delle chiavi primarie delle tabelle associate.

Scrum report

Report Scrum del Gruppo 6

Membri del gruppo: Battipaglia Edmondo, Quaranta Davide, Ragone Vincenzo,
Scassillo Umberto

Preparazione

Trello Board: [clicca qui](#)

GitHub: [clicca qui](#)

Product Backlog

User Story	Descrizione
US-01	Come utente voglio aggiungere una traccia (titolo, autore, durata, genere, anno) per arricchire la mia libreria musicale
US-02	Come utente voglio modificare i dati di una traccia esistente per correggere eventuali errori
US-03	Come utente voglio eliminare una traccia dalla libreria per rimuovere contenuti non più desiderati
US-04	Come utente voglio visualizzare l'elenco di tutte le tracce con i relativi dettagli per avere una panoramica della libreria
US-05	Come utente voglio creare una playlist con un nome identificativo per organizzare le mie tracce preferite
US-06	Come utente voglio aggiungere tracce a una playlist per personalizzarne il contenuto
US-07	Come utente voglio rimuovere tracce da una playlist per mantenerne il contenuto aggiornato

US-08	Come utente voglio eliminare una playlist per rimuovere raccolte non più utili
US-09	Come utente voglio visualizzare il contenuto di una playlist per sapere quali tracce contiene
US-10	Come utente voglio avviare la riproduzione di una traccia singola per ascoltarla
US-11	Come utente voglio avviare la riproduzione di una playlist per ascoltare le tracce in sequenza
US-12	Come utente voglio mettere in pausa e riprendere la riproduzione per gestire l'ascolto
US-13	Come utente voglio saltare alla traccia successiva o precedente per navigare nella playlist
US-14	Come utente voglio scegliere la modalità di riproduzione (sequenziale, shuffle, loop) per variare l'esperienza d'ascolto
US-15	Come utente voglio modificare la playlist durante la riproduzione (aggiungendo o rimuovendo tracce) per aggiornare l'ascolto in tempo reale
US-16	Come utente voglio vedere lo stato corrente della riproduzione (traccia in corso, avanzamento, modalità) nell'interfaccia per essere sempre informato
US-17	Come utente voglio annullare l'ultima operazione di aggiunta o rimozione di una traccia per correggere errori accidentali
US-18	Come utente voglio vedere nella homepage le tracce riprodotte più frequentemente per accedere rapidamente ai miei preferiti

US-19	Come utente voglio vedere nella homepage le playlist riprodotte più frequentemente per un accesso rapido
US-20	Come utente voglio generare automaticamente una playlist per genere per ascoltare musica coerente con il mio umore
US-21	Come utente voglio generare automaticamente una playlist per anno per rivivere un'epoca musicale
US-22	Come utente voglio aggiungere tag visivi alle tracce (preferito, chill, allenamento) per arricchire la visualizzazione e facilitare i filtri
US-23	Come utente voglio usare i tag come criterio per la generazione automatica di playlist per creare raccolte personalizzate
US-24	Come utente voglio riordinare le tracce in una playlist in qualsiasi momento, anche durante la riproduzione, per personalizzare l'ordine di ascolto
US-25	Come utente voglio visualizzare al riavvio dell'app le tracce inserite nelle precedenti sessioni in modo da non doverle reinserire ogni volta che apro l'app.

Sprint 1

1.1 Planning

La velocità iniziale del progetto è stata stimata a 19 story points, calcolata considerando la capacità del team di 4 membri con un impegno atteso di circa 8 ore settimanali a testa, tenuto conto della fase iniziale e del tempo necessario per la configurazione degli strumenti e dell'ambiente di sviluppo.

In fase di sprint planning, il team ha selezionato dal product backlog le seguenti user stories, per un totale di 19 story points:

User Story	Story Points
US1 - Aggiunta Traccia	4 SP
US2 - Modifica Traccia	4 SP
US3 - Rimozione Traccia	4 SP
US4 - Visualizza elenco Tracce	3 SP
US25 - Persistenza Dati	4 SP

La scelta di queste user stories riflette la priorità assegnata alle funzionalità di base della gestione delle tracce, in linea con gli High Priority Epics definiti nel product backlog. La US25 sulla persistenza dei dati è stata inclusa fin dal primo sprint in quanto considerata fondamentale per garantire che le funzionalità sviluppate possano essere testate in modo significativo e che i dati sopravvivano al riavvio dell'applicazione.

1.2 Review

Le user story completate corrispondono esattamente a quelle pianificate (sez. 1.1). Nessuna user story è stata rifiutata dal Product Owner: tutte le storie selezionate per lo sprint sono state accettate.

La velocità del progetto misurata al termine dello sprint è di 21 SP, superiore alla stima iniziale di 19 SP. Questo risultato è stato reso possibile dalla decisione del team di anticipare l'introduzione del supporto ai tag visivi (previsti nelle Medium Priority Epics), al fine di evitare potenziali refactoring futuri sul database e sulle classi già implementate.

Problemi riscontrati durante la review

Nel corso della demo al Product Owner è stato identificato un difetto relativo alla US1 e US2: il campo dedicato all'anno di pubblicazione della traccia non era soggetto ad alcuna validazione dell'input, consentendo all'utente di inserire valori non numerici. Tale condizione causava il lancio di un'eccezione non gestita, con conseguente comportamento anomalo dell'applicazione. Il problema è stato registrato come technical debt da risolvere nel corso dello sprint successivo.

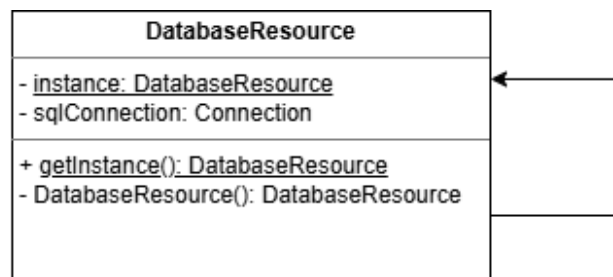
Pattern utilizzati

Durante il primo sprint il team ha adottato due design pattern: Singleton e Observer.

Singleton - DatabaseResource e ConcreteLibrary

Nel caso di DatabaseResource, l'obiettivo è garantire che esista una singola istanza della connessione al database PostgreSQL per tutta la durata dell'applicazione.

Lo stesso pattern è stato applicato a ConcreteLibrary, che rappresenta la libreria musicale dell'applicazione. Avere un'unica istanza condivisa della libreria è fondamentale per garantire la consistenza dei dati tra i diversi componenti dell'applicazione: qualsiasi modifica alle tracce viene effettuata sempre sullo stesso oggetto, evitando disallineamenti tra la vista e il modello.



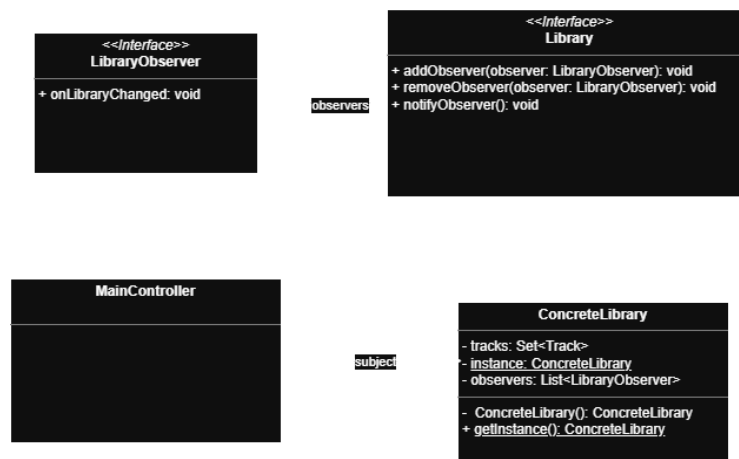
Observer - Library, LibraryObserver, ConcreteLibrary, MainController

Il pattern Observer è stato introdotto per disaccoppiare la logica di gestione della libreria musicale dalla logica di aggiornamento dell'interfaccia grafica.

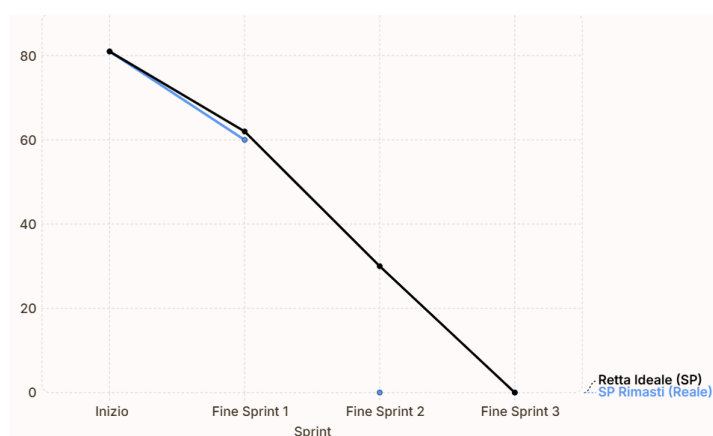
I ruoli del pattern sono distribuiti come segue:

- **Library (Subject):** interfaccia che definisce i metodi corrispondenti rispettivamente alle operazioni *attach*, *detach* e *notify* del pattern.
- **LibraryObserver (Observer):** interfaccia che definisce il metodo `onLibraryChanged()`, invocato ogni volta che lo stato della libreria viene modificato.
- **ConcreteLibrary (ConcreteSubject):** implementazione di **Library** che mantiene la lista degli observer registrati e li notifica automaticamente ad ogni operazione di aggiunta, modifica o rimozione di una traccia.
- **MainController (ConcreteObserver):** il controller principale dell'interfaccia grafica, che implementa **LibraryObserver** e aggiorna la vista in risposta alle notifiche ricevute.

Questo approccio permette a **ConcreteLibrary** di non avere alcuna dipendenza diretta verso il layer di presentazione, rispettando il principio di separazione delle responsabilità e rendendo il modello indipendente e più facilmente testabile.



Burndown Chart



Il grafico mostrato sopra rappresenta il Burndown Chart relativo all'avanzamento del primo sprint. La curva reale mostra la riduzione degli story points rimanenti nel corso dello sprint, mentre la retta nera rappresenta l'andamento ideale atteso, calcolato sulla base della complessità totale delle user story.

1.3 Retrospective

La retrospective è stata condotta seguendo il formato del diagramma a stella (*starfish diagram*), focalizzandosi esclusivamente sul processo di lavoro del team.

Start (things to start doing)

- Iniziare ad effettuare commit sin dalle prime fasi di sviluppo, con commit frequenti e granulari, per ridurre i rischi di merge tardivi.

More of (more things to do)

- Eseguire Code Review prima di procedere al merge sul branch principale.
- Mantenere una convenzione rigorosa per messaggi di commit che siano auto-esplicativi.

Keep Doing

- Rispettare la Definition of Done concordata prima di chiudere una task.
- Mantenere una comunicazione frequente e diretta all'interno del team.

Less of

- Lavoro in isolamento sulle interfacce condivise senza concordare prima con i membri del gruppo.
- Scrittura di messaggi di commit generici, o non informativi.

Stop (things to stop doing)

- Effettuare commit direttamente sul branch principale.
- Iniziare task non ancora inseriti e assegnati.

Sprint 2

2.1 Planning

La velocità di riferimento per il secondo sprint è stata ricavata dalla velocità misurata al termine del primo sprint, pari a **21 story points**. Il team ha quindi selezionato dal product backlog un insieme di user stories compatibile con questa capacità stimata.

Tuttavia, considerando che il team ha acquisito maggiore familiarità con il problema e con l'architettura del progetto, oltre a una più efficace organizzazione del lavoro maturata nel corso del primo sprint, si è deciso di aumentare il carico pianificato rispetto alla velocità misurata, selezionando un totale di **36 story points**:

User Story	Story Points
US5 - Creazione playlist	4 SP
US6 - Aggiunta tracce a playlist	4 SP
US7 - Rimozione tracce da playlist	4 SP
US8 - Eliminazione playlist	4 SP
US9 - Visualizzazione contenuto playlist	3 SP
US10 - Riproduzione singola traccia	4 SP
US12 - Gestione ascolto	3 SP
US15 - Modifica playlist	4 SP
US16 - Visualizzazione stato corrente riproduzione	2 SP
US22 - Aggiunta tag tracce	4 SP

La scelta di queste user stories segue la priorità definita nel product backlog e completa la copertura degli High Priority Epics relativi alla gestione delle playlist e alla riproduzione, avviando al contempo l'implementazione di alcune funzionalità dei Medium Priority Epics come la gestione dei tag (US22). Quest'ultima è stata anticipata rispetto alla sua priorità originale in quanto strettamente correlata al modello dati già definito nel primo sprint, limitando così il rischio di refactoring futuro.

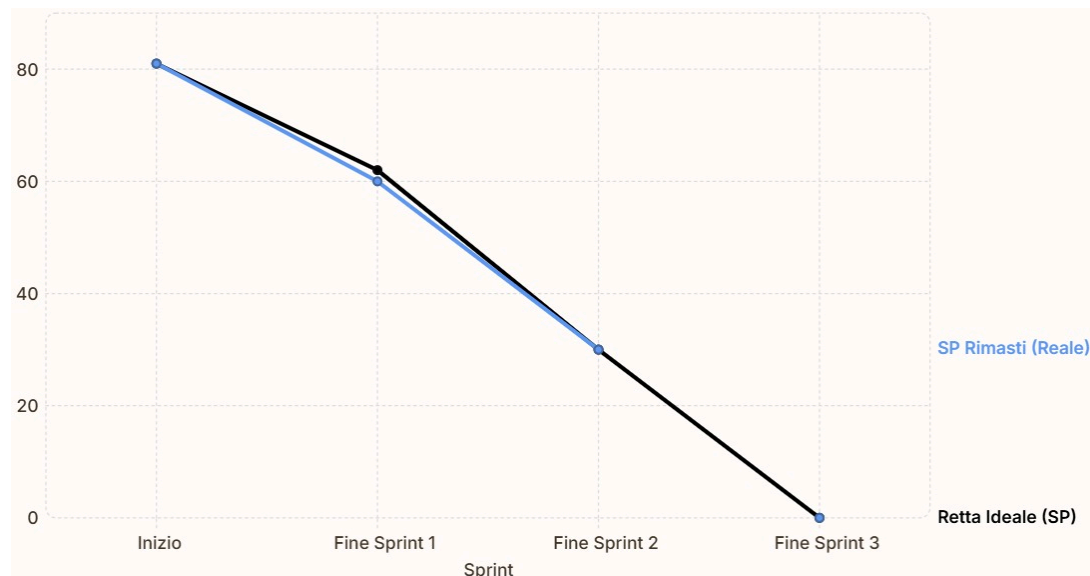
2.2 Review

Le user story completate corrispondono esattamente a quelle pianificate (sez. 2.1). Nessuna user story è stata rifiutata dal Product Owner: tutte le storie selezionate per lo sprint sono state accettate.

In aggiunta alle attività pianificate, il team ha provveduto alla correzione di alcuni difetti individuati nelle user story del primo sprint. In particolare, sono stati risolti i bug relativi a US1 - Aggiunta Traccia e US2 - Modifica Traccia, già segnalati durante la review del primo sprint. Questa attività, seppur non pianificata esplicitamente nel backlog dello sprint, è stata ritenuta prioritaria dal team al fine di consolidare la stabilità delle funzionalità già rilasciate prima di procedere con lo sviluppo delle nuove.

La velocità del progetto misurata al termine dello sprint è di 36 SP, in linea con quanto pianificato e significativamente superiore alla velocità del primo sprint (21 SP). Questo incremento è attribuibile a una più efficace organizzazione del lavoro e gestione del tempo da parte del team, maturata nel corso del primo sprint, unitamente a una maggiore familiarità con il problema e con le scelte architetturelle già effettuate, che ha permesso di ridurre significativamente i tempi di progettazione delle nuove funzionalità.

Burndown Chart



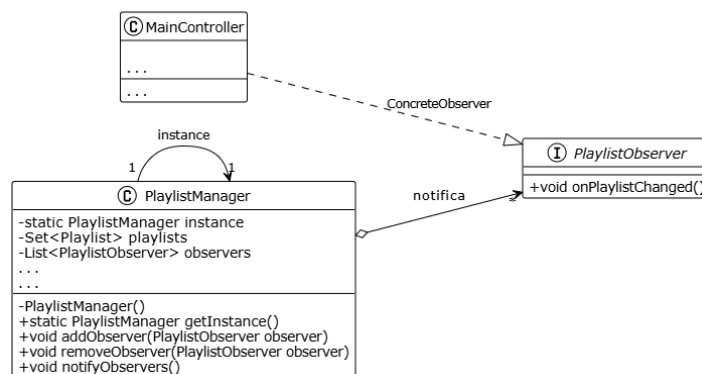
Il grafico mostrato sopra rappresenta il Burndown Chart relativo all'avanzamento del secondo sprint. La curva reale mostra la riduzione degli story points rimanenti nel corso dello sprint e risulta sostanzialmente allineata alla retta nera che rappresenta l'andamento ideale atteso, a conferma di una distribuzione del lavoro uniforme ed equilibrata nel corso dello sprint.

Pattern utilizzati

Singleton e Observer - PlaylistManager, PlaylistObserver e MainController

Analogamente a quanto fatto con ConcreteLibrary nel primo sprint, il pattern Singleton è stato applicato anche a PlaylistManager, la classe responsabile della gestione delle playlist. L'obiettivo è il medesimo: garantire un punto di accesso globale e univoco allo stato delle playlist, evitando inconsistenze tra i diversi componenti dell'applicazione che necessitano di interagire con esse.

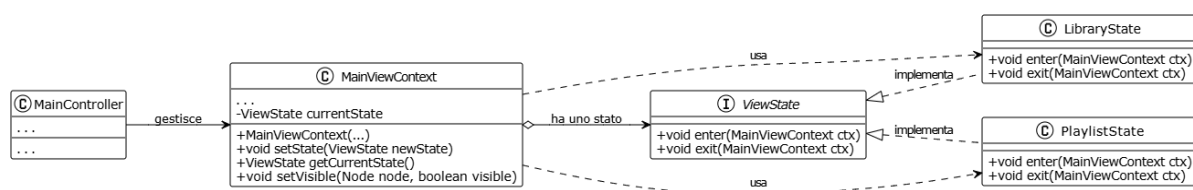
Il pattern Observer è stato esteso anche al dominio delle playlist, seguendo la stessa struttura già adottata per la libreria musicale. L'interfaccia PlaylistObserver definisce il metodo onPlaylistChanged(), invocato automaticamente da PlaylistManager tramite notifyObservers() ogni volta che lo stato delle playlist viene modificato, ovvero in seguito a operazioni di creazione, eliminazione, rinomina, aggiunta o rimozione di tracce. Il MainController si registra come observer e provvede ad aggiornare la vista in risposta a tali notifiche, mantenendo così il modello completamente disaccoppiato dal layer di presentazione.



State - ViewState, LibraryState, PlaylistState, MainViewContext e MainController

Il pattern State è stato introdotto per gestire i due modi principali in cui può trovarsi la vista principale dell'applicazione: la visualizzazione della libreria musicale e la visualizzazione del contenuto di una playlist. Questi due contesti richiedono la presenza di controlli diversi nell'interfaccia grafica, e gestire questa logica tramite una serie di controlli condizionali sparsi nel MainController avrebbe portato a codice poco leggibile e difficilmente manutenibile.

Questo approccio centralizza la logica di aggiornamento dell'interfaccia all'interno delle classi di stato, rendendo il MainController più snello e le eventuali modifiche future all'interfaccia più localizzate e controllate.



2.3 Retrospective

La retrospective è stata condotta seguendo il formato del diagramma a stella (*starfish diagram*), focalizzandosi esclusivamente sul processo di lavoro del team.

Start (things to start doing)

- Iniziare ad effettuare commit sin dalle prime fasi di sviluppo, con commit frequenti e granulari, per ridurre i rischi di merge tardivi.

More of (more things to do)

- Eseguire Code Review prima di procedere al merge sul branch principale.
- Mantenere una convenzione rigorosa per messaggi di commit che siano auto-esplicativi.

Keep Doing

- Rispettare la Definition of Done concordata prima di chiudere una task.
- Mantenere una comunicazione frequente e diretta all'interno del team.

Less of

- Lavoro in isolamento sulle interfacce condivise senza concordare prima con i membri del gruppo.
- Scrittura di messaggi di commit generici, o non informativi.

Stop (things to stop doing)

- Effettuare commit direttamente sul branch principale.
- Iniziare task non ancora inseriti e assegnati.

Sprint 3

3.1 Planning

La velocità di riferimento per il terzo sprint è stata ricavata dalla velocità misurata al termine del secondo sprint, pari a 36 story points. Tenendo conto della capacità del team e dell'esperienza accumulata nei due sprint precedenti, il team ha selezionato dal product backlog un insieme di user stories per un totale di 26 story points:

User Story	Story Points
US11 - Riproduzione playlist	4 SP
US13 - Navigazione tracce nella Playlist	2 SP
US14 - Scelta modalità di riproduzione	1 SP
US17 - Annullamento operazione di aggiunta o rimozione	2 SP
US18 - Visualizzazione tracce più frequenti	3 SP
US19 - Visualizzazione playlist più frequenti	3 SP
US20 - Generazione playlist per genere	3 SP
US21 - Generazione playlist per anno	3 SP
US23 - Uso tag per generazione playlist	3 SP
US24 - Riordinamento delle tracce	2 SP

Il carico pianificato per questo sprint è inferiore alla velocità misurata nel secondo sprint. Questa scelta è stata deliberata: trattandosi dell'ultimo sprint, il team ha preferito adottare un approccio conservativo per garantire che tutte le user stories selezionate vengano completate rispettando la Definition of Done, senza compromettere la qualità del prodotto finale. Le user stories scelte completano la copertura degli High Priority Epics relativi alla riproduzione e introducono le funzionalità dei Medium Priority Epics, tra cui la generazione automatica di playlist, la visualizzazione delle tracce e playlist più frequenti e la gestione dei tag.

3.2 Review

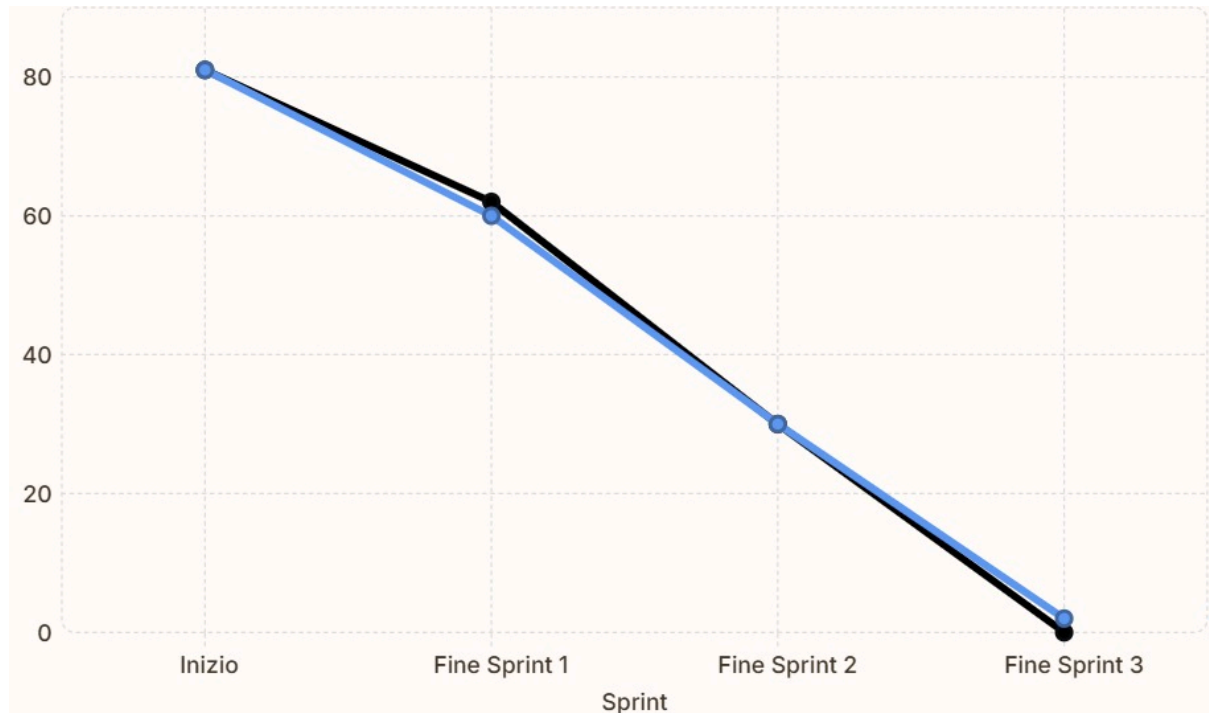
Le User Story completate al termine dello sprint sono:

- US11 – Riproduzione Playlist
- US13 – Navigazione Tracce nella Playlist
- US14 – Scelta Modalità di Riproduzione
- US17 – Annullamento Operazione di Aggiunta o Rimozione
- US18 – Visualizzazione Tracce più Frequenti
- US19 – Visualizzazione Playlist più Frequenti
- US20 – Generazione Playlist per Genere
- US21 – Generazione Playlist per Anno
- US23 – Uso Tag per Generazione Playlist

La quasi totalità delle user stories pianificate è stata completata e accettata dal Product Owner. L'unica eccezione è rappresentata da US24 – Riordinamento delle Tracce, che non è stata portata a termine entro la fine dello sprint. La funzionalità di riordinamento era stata parzialmente implementata a livello di interfaccia grafica, tuttavia le acceptance criteria concordate richiedevano che le modifiche all'ordine delle tracce fossero persistenti, ovvero mantenute anche a seguito del riavvio dell'applicazione. L'implementazione della relativa logica di persistenza non è stato possibile completarla nei tempi previsti, pertanto la user story non è stata accettata dal Product Owner in quanto non conforme alla Definition of Done. La US24 rimane quindi nel product backlog come non completata.

La velocità del progetto misurata al termine dello sprint è di 24 SP, leggermente inferiore ai 26 pianificati a causa della mancata completamento di US24 (2 SP).

Burndown Chart



Il grafico mostrato sopra rappresenta il Burndown Chart relativo all'avanzamento del terzo sprint. La curva reale mostra un andamento generalmente in linea con la retta ideale nel corso dello sprint, discostandosi leggermente verso la fine a causa della mancata completamento di US24 – Riordinamento delle Tracce. I 2 story points non completati hanno determinato un piccolo scostamento finale rispetto all'andamento ideale atteso, lasciando la curva reale al di sopra dello zero al termine dello sprint. Questo scostamento è ritenuto accettabile in quanto circoscritto a una singola user story di bassa priorità, il cui mancato completamento non ha compromesso la qualità né la funzionalità complessiva del prodotto rilasciato.

Pattern utilizzati

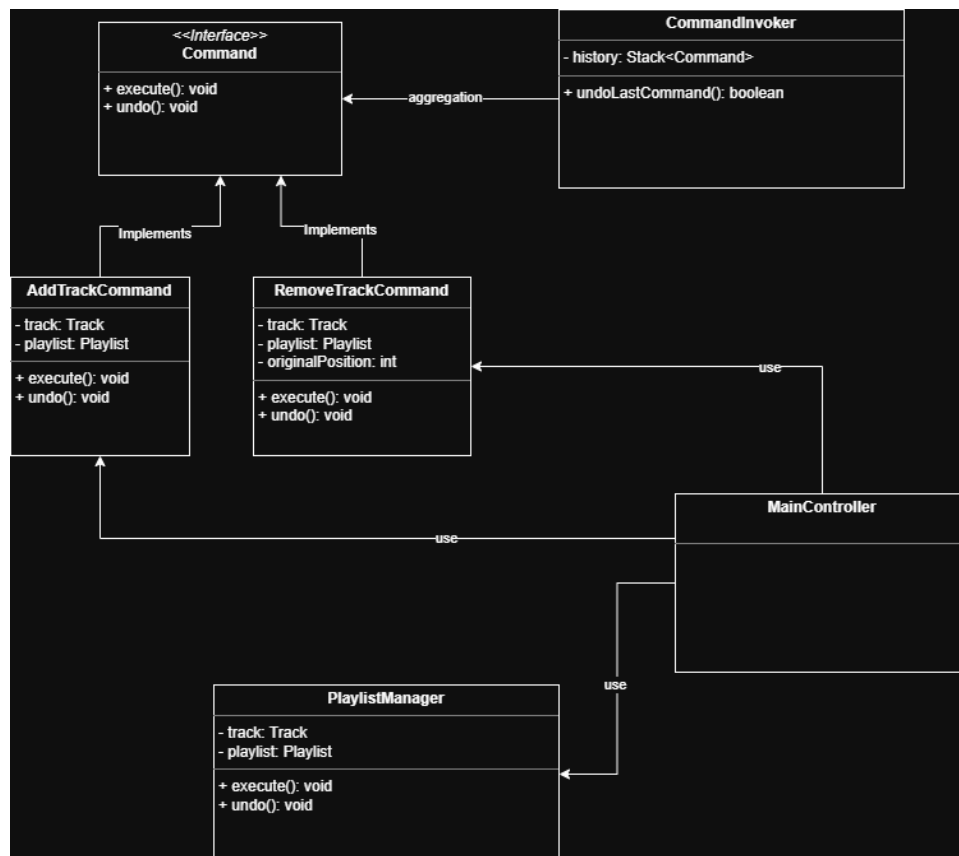
Command - Command, CommandInvoker, AddTrackCommand, RemoveTrackCommand

Il pattern Command è stato introdotto per implementare la funzionalità di annullamento delle operazioni di aggiunta e rimozione di tracce da una playlist, corrispondente alla US17. L'obiettivo è disaccoppiare il codice che richiede l'esecuzione di un'operazione dal codice che la esegue effettivamente, incapsulando ogni operazione in un oggetto autonomo che ne conosce sia la logica di esecuzione che quella di annullamento.

I ruoli del pattern sono distribuiti come segue:

- **Command** (Command): interfaccia che definisce i metodi `execute()` e `undo()`, che ogni comando concreto deve implementare.

- **CommandInvoker** (Invoker): classe responsabile dell'esecuzione dei comandi e della gestione della loro cronologia tramite uno stack. Il metodo `executeCommand()` esegue il comando e lo memorizza nello stack, mentre `undoLastCommand()` estrae l'ultimo comando dallo stack e ne invoca il metodo `undo()`, restituendo `false` nel caso in cui non vi siano operazioni da annullare.
- **AddTrackCommand** (ConcreteCommand): incapsula l'operazione di aggiunta di una traccia a una playlist. Il metodo `execute()` delega l'operazione al `PlaylistManager`, mentre `undo()` ne esegue la rimozione.
- **RemoveTrackCommand** (ConcreteCommand): incapsula l'operazione di rimozione di una traccia da una playlist. Prima di eseguire la rimozione, memorizza la posizione originale della traccia all'interno della playlist in modo da poterla ripristinare correttamente durante l'operazione di undo, ricostruendo l'ordine originale delle tracce nella lista.

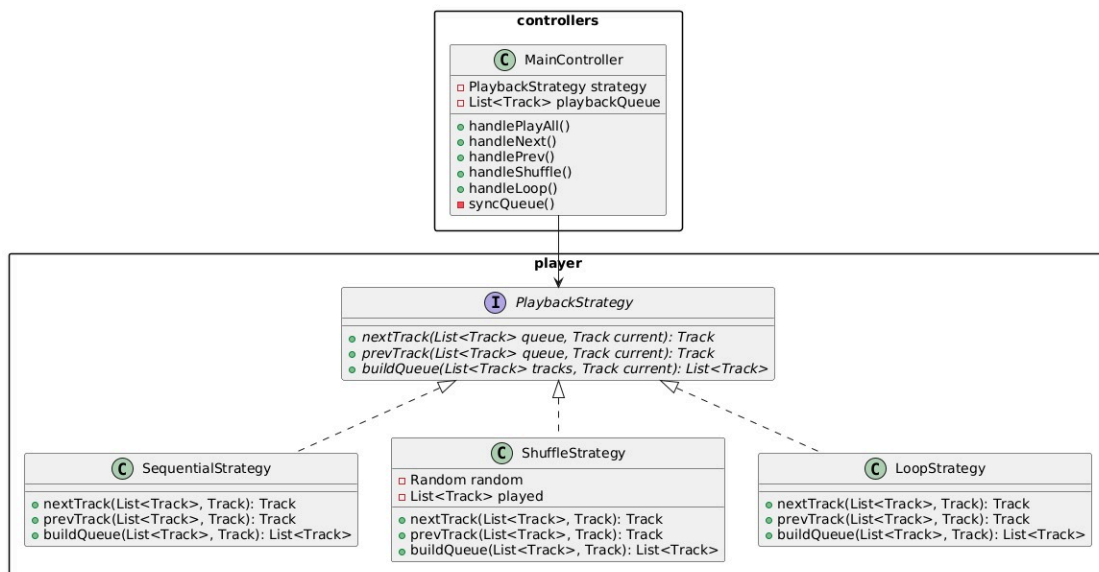


Strategy - PlaybackStrategy, SequentialStrategy, ShuffleStrategy, LoopStrategy, MainController

Il pattern Strategy è stato adottato per gestire le tre modalità di riproduzione previste dall'applicazione: sequenziale, casuale e in loop. Ogni modalità differisce nel modo in cui viene determinata la traccia successiva o precedente durante la riproduzione, e gestire questa logica tramite costrutti condizionali all'interno del player avrebbe prodotto codice rigido e difficilmente estendibile.

I ruoli del pattern sono distribuiti come segue:

- **PlaybackStrategy** (Strategy): interfaccia che definisce i metodi `nextTrack()`, `prevTrack()` e `buildQueue()`, che ogni strategia concreta deve implementare per determinare rispettivamente la traccia successiva, quella precedente e la coda di riproduzione.
- **SequentialStrategy** (ConcreteStrategy): implementa la riproduzione sequenziale. Le tracce vengono riprodotte nell'ordine in cui compaiono nella coda; `nextTrack()` restituisce la traccia successiva nell'indice e `prevTrack()` quella precedente, restituendo `null` quando si raggiunge il limite della coda.
- **LoopStrategy** (ConcreteStrategy): implementa la riproduzione in loop sulla traccia corrente. Sia `nextTrack()` che `prevTrack()` restituiscono sempre la traccia corrente, indipendentemente dalla posizione nella coda.
- **ShuffleStrategy** (ConcreteStrategy): implementa la riproduzione casuale. La strategia mantiene internamente una lista delle tracce già ascoltate, in modo da non riproporre la stessa traccia fino all'esaurimento della coda. Quando tutte le tracce sono state riprodotte, la lista viene resettata e la riproduzione ricomincia. Il metodo `prevTrack()` naviga a ritroso nella cronologia delle tracce ascoltate, mentre `buildQueue()` genera una coda mescolata posizionando la traccia corrente in prima posizione.



3.3 Retrospective

La retrospective conclusiva è stata condotta seguendo il formato del diagramma a stella (*starfish diagram*), con una riflessione che abbraccia sia l'andamento dell'ultimo sprint che l'intero percorso del progetto.

Start

In retrospettiva, il team riconosce che sarebbe stato utile dedicare maggiore attenzione alla definizione delle acceptance criteria fin dalle prime fasi del progetto, in particolare per le user stories che coinvolgono la persistenza dei dati. Un'analisi più

approfondita in fase di sprint planning avrebbe permesso di stimare con maggiore precisione la complessità di alcune storie, evitando situazioni come quella di US24, la cui mancata completamento è stata determinata proprio da un requisito di persistenza non adeguatamente considerato in fase di pianificazione.

More of

Il team intende consolidare ulteriormente la pratica della stima degli story points, cercando di tenere in maggiore considerazione le dipendenze tecniche nascoste che possono emergere durante lo sviluppo, come la necessità di gestire la persistenza di funzionalità apparentemente semplici.

Keep doing

In questo sprint il team ha raggiunto un livello di collaborazione e organizzazione notevolmente migliorato rispetto alle sprint precedenti. La gestione di GitHub è risultata efficace, con un utilizzo corretto dei branch e dei commit, e la comunicazione interna al gruppo ha permesso di coordinare il lavoro dei vari membri senza generare conflitti significativi nel codice.

Less of

Il team ha quasi del tutto eliminato la tendenza a lavorare in isolamento su componenti condivise senza un preventivo allineamento con gli altri membri, problema emerso nelle prime sprint e progressivamente ridotto nel corso del progetto.

Stop

Il team ha definitivamente abbandonato le pratiche problematiche identificate nelle sprint precedenti, tra cui i commit diretti sul branch principale e l'utilizzo di messaggi di commit generici. Queste abitudini, corrette già a partire dal secondo sprint, non hanno rappresentato criticità nell'ultima fase del progetto.

4 Diagramma delle Classi

vedi file png associati